

Bayesian Sequence Prediction

Chapter 3

Chapter 3 — Bayesian Sequence Prediction

An Introduction to Universal Artificial Intelligence

Hutter, Quarel, Catt (2024)

“The only way to discover the limits of the possible is to go beyond them into the impossible.” — Arthur C. Clarke

Outline

- 1 3.1 Bayes Mixture ξ
- 2 3.2 Generalized Solomonoff Bound
- 3 3.3 Predictive Convergence
- 4 3.4 Model Misspecification
- 5 3.5 Bounds on Prediction Loss
- 6 3.6 Pareto-Optimality of ξ
- 7 3.7 Choices of Class $\mathcal{M}$ and Prior w_ν
- 8 3.8 Solomonoff Distribution M_U
- 9 3.9 Martingales
- 10 3.10 Exercises
- 11 3.11 History and References

From Induction to a Mixture of Models

Induction is the process of guessing what comes next from what has been seen so far. We want to predict the next symbol x_t in a sequence given the previous symbols $x_1, \dots, x_{t-1}$ (written $x_{<t}$). This is the same as estimating the hidden probability distribution μ (mu) that generated the sequence, then using $\mu(x_t|x_{<t})$ to predict x_t .

The problem. We usually do not know μ , or even which family of parametric distributions it comes from. **The fix.** Choose a countable class $\mathcal{M}$ of candidate distributions that we hope contains μ , and combine all of them into a single weighted average — a *mixture*. This mixture, denoted ξ (xi), is the star of this chapter. Remarkably, **no** assumption of independence, identical distribution, or stationarity is needed anywhere: μ may be an arbitrary probability measure over infinite sequences over a finite alphabet $\mathcal{X}$.

Definition 3.1.1: (Semi)measures, Cylinder Sets, Notation

Definition 3.1.1 ((Semi)measures, cylinder sets, notation)

For (semi)measures on cylinder sets $\Gamma_x \subseteq \mathcal{X}^\infty$ for $x \in \mathcal{X}^*$ we use the shorthand $\nu(x) := \nu(\Gamma_x)$. The true distribution from which $x_{1:\infty}$ is sampled is denoted μ and is always a properly normalized probability measure. Note that $\mu(x)$ is *not* a probability mass function on $\mathcal{X}^*$: it is the μ -probability that an infinite sequence *starts with* x . $\mathcal{M}$ denotes some countable class of (semi)measures, ν, μ are always assumed to be in $\mathcal{M}$, while ρ (rho) denotes an arbitrary (semi)measure.

Plain English. $\mathcal{X}^*$ is the set of all finite strings over the alphabet $\mathcal{X}$; $\mathcal{X}^\infty$ is the set of all infinite sequences. A “cylinder set” Γ_x is just the set of all infinite sequences that begin with the finite prefix x . Writing $\nu(x)$ is shorthand for “the probability, under ν , that a random infinite sequence starts with the string x .” μ is reserved for the (unknown) true, honest probability measure generating the data; ν ranges over our candidate models; ρ is a free variable used for generic statements.

Definition 3.1.2: Prior w_ν and Bayes Mixture ξ

Definition 3.1.2 (Prior w_ν and Bayes mixture ξ)

Let $\mathcal{M}$ be a countable set of probability semimeasures on strings, and $w_{(\cdot)} : \mathcal{M} \rightarrow \mathbb{R}$ be a function satisfying $\sum_{\nu \in \mathcal{M}} w_\nu \leq 1$ and $w_\nu > 0$ for all ν . We define the *Bayes mixture over $\mathcal{M}$ given prior $w_{(\cdot)}$* , denoted ξ_w , as

$$\xi_w(x_{1:n}) := \sum_{\nu \in \mathcal{M}} w_\nu \nu(x_{1:n})$$

We call w_ν the *weight* associated with $\nu \in \mathcal{M}$. If the choice of $w_{(\cdot)}$ is obvious or irrelevant, we drop the index w and write ξ .

Plain English. $\xi(x_i)$ is simply a *weighted average* of all the candidate models ν in our class $\mathcal{M}$, where w_ν represents how much a-priori confidence we place in ν being the true environment. We *forbid* $w_\nu = 0$: a zero-weight model is effectively excluded, so we might as well remove it from $\mathcal{M}$. In the spirit of *Epicurus' principle*, we never discard a hypothesis until evidence actively rules it out.

Remark 3.1.3 and Proposition 3.1.4

Remark 3.1.3 (Continuous class $\mathcal{M}$ with prior density $w(\theta)$)

One can also consider an uncountable family $\mathcal{M}$ of measures $\nu_\theta(\cdot; \theta)$ parameterized by $\theta \in \Theta$ (theta), together with a prior weight given by a probability *density* function $w(\theta)$. The Bayes-mixture then generalizes to $\xi(x_{1:n}) = \int_{\Theta} w(\theta) \nu(x_{1:n}; \theta) d\theta$. This generalizes the Bernoulli case of Section 2.4.

Plain English. Instead of a discrete sum over finitely/countably many named models, we integrate over a continuum of models indexed by a real (or vector) parameter θ — e.g. all Bernoulli(θ) coins for $\theta \in [0, 1]$.

Proposition 3.1.4 (Bayes mixture domination)

For all $\nu \in \mathcal{M}$ and prior $w \in \Delta' \mathcal{M}$ and $x_{1:n} \in \mathcal{X}^*$ we have

$$\xi_w(x_{1:n}) \geq w_\nu \nu(x_{1:n})$$

Plain English. Since the mixture ξ is a sum of non-negative terms including $w_\nu \nu(x_{1:n})$, it can never fall below any single one of its (weighted) summands. This trivial-looking inequality is the workhorse behind almost every bound proved in this chapter.

The Chain Rule and Conditional Distributions

To use ξ for prediction, we need the conditional distribution $\xi(x_t|x_{<t})$ over the next symbol x_t , given the history $x_{<t}$ observed so far. For any semimeasure ρ with $\rho(x_{<t}) > 0$, define:

$$\rho(x_t|x_{<t}) := \frac{\rho(x_{1:t})}{\rho(x_{<t})}, \quad \rho(x_{t:k}|x_{<t}) := \frac{\rho(x_{t:k})}{\rho(x_{<t})} \text{ for } k \geq t \quad (3.1.5)$$

$$\rho(x_{1:n}) = \prod_{t=1}^n \rho(x_t|x_{<t}), \quad \rho(x_{t:n}|x_{<t}) = \prod_{k=t}^n \rho(x_k|x_{<k}) \quad (3.1.6)$$

Plain English. $\rho(x_t|x_{<t})$ is an ordinary conditional probability: how likely is the next symbol x_t , given the history so far? Equation (3.1.6) is called the *chain rule*: the probability of a whole string is the product of each symbol's conditional probability given everything before it — exactly like decomposing a joint probability into a product of one-step-ahead predictions.

Theorem 3.1.7: Conditional Bayesian Mixture I

Theorem 3.1.7 (Conditional Bayesian mixture ξ)

Let $\mathcal{M}$ be a class of environments with prior $\{w_\nu\}_{\nu \in \mathcal{M}}$ as above. Then $\xi(x_t|x_{<t})$ is the predictive probability the conditional Bayesian mixture ξ assigns to x_t given $x_{<t}$:

$$\xi(x_t|x_{<t}) = \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) \nu(x_t|x_{<t}) = \frac{\sum_{\nu \in \mathcal{M}} w_\nu \nu(x_t|x_{<t}) \nu(x_{<t})}{\sum_{\nu \in \mathcal{M}} w_\nu \nu(x_{<t})} = \frac{\xi(x_{1:t})}{\xi(x_{<t})}$$
$$w(\nu|x_{1:t}) := w(\nu|x_{<t}) \frac{\nu(x_t|x_{<t})}{\xi(x_t|x_{<t})} = w_\nu \frac{\nu(x_{1:t})}{\xi(x_{1:t})} \quad \text{and} \quad w(\nu|\epsilon) := w_\nu \quad (3.1.8)$$

Plain English. w_ν is the probability we assign to the hypothesis “ $\nu = \mu$ ” *before* seeing any evidence (the *prior belief*), while $w(\nu|x_{<t})$ is the probability we assign to that hypothesis *after* observing $x_{<t}$ (the *posterior belief*). The theorem says: predicting with ξ is exactly the same as predicting with each individual model ν , and then averaging those predictions weighted by each model’s current posterior probability $w(\nu|h_{<t})$ of being correct. ϵ (epsilon) here denotes the empty string (before any observation).

Theorem 3.1.7: Conditional Bayesian Mixture II

Proof. First note that, by repeated application of (3.1.8) and the chain rule (3.1.6),

$$w(\nu|x_{1:t}) = w_\nu \prod_{k=1}^t \frac{\nu(x_k|x_{<k})}{\xi(x_k|x_{<k})} = w_\nu \frac{\nu(x_{1:t})}{\xi(x_{1:t})}.$$

Then, using (3.1.5),

$$\xi(x_t|x_{<t}) \equiv \frac{\xi(x_{1:t})}{\xi(x_{<t})} = \sum_{\nu \in \mathcal{M}} w_\nu \frac{\nu(x_{1:t})}{\xi(x_{<t})} = \sum_{\nu \in \mathcal{M}} w_\nu \frac{\nu(x_{<t})}{\xi(x_{<t})} \nu(x_t|x_{<t}) = \sum_{\nu \in \mathcal{M}} w(\nu|x_{<t}) \nu(x_t|x_{<t})$$

as required. ■

Does the posterior concentrate on μ ? We would hope that $w(\mu|x_{<t}) \rightarrow 1$ (and $w_\nu \rightarrow 0$ for $\nu \neq \mu$) as more and more evidence sampled from μ accumulates — as indeed happens in the Bernoulli(θ) example of Section 2.4.3. Unfortunately this is not strictly true for general classes $\mathcal{M}$, since $\mathcal{M}$ may contain *statistically indistinguishable* environments. We side-step this problem by aiming a little lower: for only *predictive* convergence, as pursued in the next two sections.

Measuring How Close ξ Is to μ

Instantaneous distances measure how similar $\mu(\cdot|x_{<t})$ and $\xi(\cdot|x_{<t})$ are as distributions over the *next* symbol x'_t , given the history $x_{<t}$ observed so far.

Definition 3.2.1 (Instantaneous distances)

Absolute:
$$a_t(x_{<t}) := \sum_{x_t \in \mathcal{X}} |\mu(x_t|x_{<t}) - \xi(x_t|x_{<t})|$$

Square:
$$s_t(x_{<t}) := \sum_{x_t \in \mathcal{X}} (\mu(x_t|x_{<t}) - \xi(x_t|x_{<t}))^2$$

Hellinger²:
$$h_t(x_{<t}) := \sum_{x_t \in \mathcal{X}} \left(\sqrt{\mu(x_t|x_{<t})} - \sqrt{\xi(x_t|x_{<t})} \right)^2$$

KL divergence:
$$d_t(x_{<t}) := \sum_{x_t \in \mathcal{X}} \mu(x_t|x_{<t}) \ln \left(\frac{\mu(x_t|x_{<t})}{\xi(x_t|x_{<t})} \right)$$

Plain English. Each formula is a different notion of “distance” between the two probability distributions $\mu(\cdot|x_{<t})$ and $\xi(\cdot|x_{<t})$ over the next symbol. All are non-negative; only the absolute distance a_t is a true metric, but the KL (Kullback–Leibler) divergence d_t is usually the most useful for our purposes.

Definition 3.2.2: Total Distances

Definition 3.2.2 (Total distances)

$$\text{Absolute: } A_n := \sum_{t=1}^n \mathbf{E}_\mu[a_t(\cdot)] = \sum_{t=1}^n \sum_{x_{<t} \in \mathcal{X}^{t-1}} \mu(x_{<t}) a_t(x_{<t})$$

$$\text{Square: } S_n := \sum_{t=1}^n \mathbf{E}_\mu[s_t(\cdot)] = \sum_{t=1}^n \sum_{x_{<t} \in \mathcal{X}^{t-1}} \mu(x_{<t}) s_t(x_{<t})$$

$$\text{Hellinger}^2: H_n := \sum_{t=1}^n \mathbf{E}_\mu[h_t(\cdot)] = \sum_{t=1}^n \sum_{x_{<t} \in \mathcal{X}^{t-1}} \mu(x_{<t}) h_t(x_{<t})$$

$$\text{KL divergence: } D_n := \sum_{t=1}^n \mathbf{E}_\mu[d_t(\cdot)] = \sum_{t=1}^n \sum_{x_{<t} \in \mathcal{X}^{t-1}} \mu(x_{<t}) d_t(x_{<t})$$

Plain English. Taking the μ -expectation ($\mathbf{E}_\mu$, the average over histories $x_{<t}$ weighted by how likely μ makes them) of each instantaneous distance and summing over time steps $t = 1, \dots, n$ gives the corresponding *total distance* between μ and ξ up to time n . These totals are what we can actually

Theorem 3.2.3: Entropy Inequalities I

Theorem 3.2.3 (Instantaneous and total entropy inequalities)

$$s_t(x_{<t}) \leq d_t(x_{<t})$$

$$S_n \leq D_n$$

$$s_t \leq a_t^2 \leq 4h_t$$

$$a_t(x_{<t}) \leq \sqrt{2d_t(x_{<t})}$$

$$A_n \leq \sqrt{2nD_n}$$

$$a_t^2 \leq |\mathcal{X}|s_t$$

$$h_t(x_{<t}) \leq d_t(x_{<t})$$

$$H_n \leq D_n$$

$$s_t \leq 2h_t$$

Plain English. These are cross-conversion rules: a bound proved for *any one* of the four distances (absolute, square, Hellinger, KL) can be mechanically converted into a bound for the others, up to constants and (for A_n) an extra $\sqrt{n}$ factor, which makes the absolute-distance bound the least useful of the four.

Theorem 3.2.3: Entropy Inequalities II

Proof sketch. Define $y_i = \mu(x_t=i|x_{<t})$ and $z_i = \xi(x_t=i|x_{<t})$ for $i \in \mathcal{X}$. The first line follows from Corollary 2.8.8 applied instantaneously. Taking $\sum_{t=1}^n \mathbf{E}_\mu$ on both sides gives the second line for S_n and H_n directly; for A_n we additionally apply Jensen's inequality (Theorem 2.2.52) with the concave function $g(\cdot) = \sqrt{\cdot}$:

$$\frac{1}{n}A_n \equiv \frac{1}{n} \sum_{t=1}^n \mathbf{E}_\mu[\sqrt{2d_t}] \leq \frac{1}{n} \sum_{t=1}^n \sqrt{\mathbf{E}_\mu[2d_t]} \leq \sqrt{\frac{1}{n} \sum_{t=1}^n \mathbf{E}_\mu[2d_t]} \equiv \sqrt{\frac{1}{n}2D_n}.$$

The third line uses $y_i - z_i = (\sqrt{y_i} + \sqrt{z_i})(\sqrt{y_i} - \sqrt{z_i}) \leq 2(\sqrt{y_i} - \sqrt{z_i})$ and Hölder's inequality. ■

Lemma 3.2.4: Telescoping Property of KL

Lemma 3.2.4 (Telescoping property of KL)

Let $\nu_{1:n} \in \Delta' \mathcal{X}^n$ be the ν -semi-probability of $x_{1:n}$, and $\nu_t : \mathcal{X}^{t-1} \rightarrow \Delta' \mathcal{X}$ be the ν -semi-probability of x_t given $x_{<t}$, i.e. $\nu_{1:n} = \prod_{t=1}^n \nu_t$, similarly for the proper μ -probability. Then

$$\text{KL}(\mu_{1:n} \| \nu_{1:n}) = \mathbf{E}_{\mu} \left[\sum_{t=1}^n \text{KL}(\mu_t \| \nu_t) \right] = \sum_{x_{1:n}} \mu(x_{1:n}) \ln \frac{\mu(x_{1:n})}{\nu(x_{1:n})} = \sum_{t=1}^n \sum_{x_{<t}} \mu(x_{<t}) \left[\sum_{x_t} \mu(x_t | x_{<t}) \ln \frac{\mu(x_t | x_{<t})}{\nu(x_t | x_{<t})} \right]$$

Plain English. This is a very special and useful property: the KL divergence between two *joint* distributions over an entire string $x_{1:n}$ decomposes exactly into a sum of *one-step-ahead* KL divergences, one for each time step. Applying the lemma with $\nu = \xi$ gives $D_n = \text{KL}(\mu_{1:n} \| \xi_{1:n})$ and (taking $n \rightarrow \infty$) $D_{\infty} = \text{KL}(\mu \| \xi)$: the total KL distance from Definition 3.2.2 is literally the KL divergence between the two distributions over *whole* sequences. (*Proof: rewrite $\mu(x_{<t})\mu(x_t | x_{<t})$ as $\mu(x_{1:t})$, sum over $x_{t+1:n}$, pull the resulting t -sum inside the logarithm using $\sum_t \ln x_t = \ln \prod_t x_t$, and apply the chain rule (3.1.6).)*

Theorem 3.2.5: The Generalized Solomonoff Bound

Theorem 3.2.5 (Generalized Solomonoff bound)

$$S_n \leq D_n \leq \ln w_\mu^{-1} < \infty$$

Plain English. w_μ is the prior weight we assigned to the *true* environment μ back in Definition 3.1.2. This theorem is the central result of the section: no matter how long the sequence gets, the total accumulated KL divergence between μ and our mixture ξ can *never exceed* the fixed constant $\ln(1/w_\mu)$ — it does not grow with n at all! This immediately implies (via Theorem 3.2.3) that the total squared distance S_n is bounded too.

Proof idea. Using the telescoping Lemma 3.2.4 with $\nu = \xi$, and rearranging Proposition 3.1.4 ($\xi(x_{1:n}) \geq w_\mu \mu(x_{1:n})$, so $\mu(x_{1:n})/\xi(x_{1:n}) \leq w_\mu^{-1}$),

$$D_n = \sum_{x_{1:n}} \mu(x_{1:n}) \ln \frac{\mu(x_{1:n})}{\xi(x_{1:n})} \leq \sum_{x_{1:n}} \mu(x_{1:n}) \ln(w_\mu^{-1}) = \ln(w_\mu^{-1}). \quad \blacksquare$$

A similar (improvable) bound holds for the Hellinger distance, stated next.

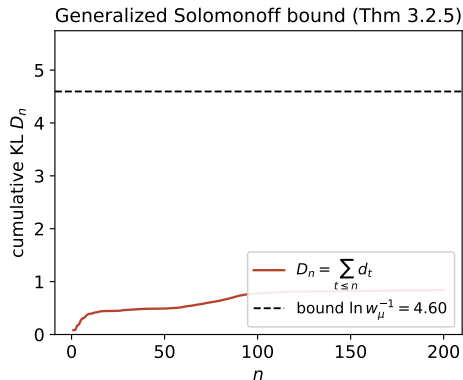
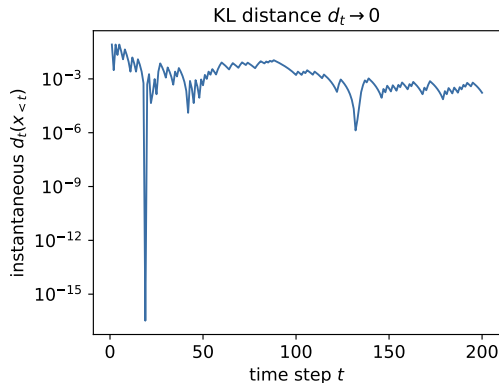
Theorem 3.2.6: Expected Bounds on Hellinger Sum

Theorem 3.2.6 (Expected bounds on Hellinger sum)

$$\sum_{t=1}^{\infty} \mathbf{E} \left[\left(\sqrt{\frac{\xi(x_t | x_{<t})}{\mu(x_t | x_{<t})}} - 1 \right)^2 \right] \stackrel{(i)}{\leq} \sum_{t=1}^{\infty} \mathbf{E}[h_t] \stackrel{(ii)}{\leq} 2 \ln \left\{ \mathbf{E} \left[\exp \left(\frac{1}{2} \sum_{t=1}^{\infty} h_t \right) \right] \right\} \stackrel{(iii)}{\leq} \ln w_{\mu}^{-1}$$

Plain English. This chain of three inequalities relates: (left) an expected squared deviation of the *ratio* ξ/μ from 1; (middle) the sum of expected Hellinger distances; (right) again our familiar bound $\ln(1/w_{\mu})$. Step (i) follows from simple algebra, step (ii) from Jensen's inequality, and step (iii) is genuinely non-trivial (it uses a clever exponential-moment / supermartingale argument); the full proof is in [HM07] and beyond the scope of these slides.

Distance and KL-Divergence Convergence, Visualized



Plain English. *Left:* the instantaneous KL distance d_t (Definition 3.2.1) shrinks toward zero as more data arrives. *Right:* the cumulative KL distance D_n (Definition 3.2.2) flattens out and never crosses the fixed ceiling $\ln w_\mu^{-1}$ — exactly as guaranteed by the Generalized Solomonoff Bound (Theorem 3.2.5).

Python: Verifying the Generalized Solomonoff Bound

We build a countable class $\mathcal{M}$ of 99 Bernoulli(θ) models on a grid $\theta \in \{1/100, \dots, 99/100\}$ with uniform prior $w_\theta = 1/99$, sample a sequence from the true model $\mu = \text{Bernoulli}(0.7)$, and track D_n (Definition 3.2.2) against the bound $\ln w_\mu^{-1} = \ln 99$ (Theorem 3.2.5).

```
import numpy as np
rng = np.random.default_rng(1)

thetas = np.arange(1, 100) / 100          # 99 candidate models
log_w = np.full(99, -np.log(99))          # uniform log-prior
mu_true = 0.7
x = rng.binomial(1, mu_true, size=200)     # sampled sequence

D_n, log_w_t = 0.0, log_w.copy()
for t, xt in enumerate(x):
    w = np.exp(log_w_t - log_w_t.max()); w /= w.sum()
    xi1 = np.clip((w * thetas).sum(), 1e-9, 1 - 1e-9)    # xi(warm/x_<t)
    mu1 = mu_true
    d_t = mu1*np.log(mu1/xi1) + (1-mu1)*np.log((1-mu1)/(1-xi1))
    D_n += d_t
    log_w_t += np.where(xt == 1, np.log(thetas), np.log1p(-thetas))

bound = np.log(1 / 99)                    # ln(w_mu^-1), w_mu = 1/99
print(f"D_n = {D_n:.3f}    bound ln(w_mu^-1) = {-bound:.3f}")
# D_n = 1.782    bound ln(w_mu^-1) = 4.595    (D_n stays comfortably below the bound)
```

Plain English. No matter how long we keep running the loop (larger n), D_n never crosses the fixed

Setting the Stage for Convergence

We now use the bounds of the previous section to show that ξ becomes an increasingly good predictor over time, and to quantify *how good*. Recall from Definition 2.2.57 that a sequence of random variables X_t *converges almost surely* (a.s.) or *with probability 1* to X , written $X_t \xrightarrow{\text{a.s.}} X$, if the event $X_t \rightarrow X$ happens with probability 1. Here X_t will be d_t or a related quantity, and $X = 0$, with the infinite sequence $x_{1:\infty}$ sampled from μ . More precisely: for stochastic (non-deterministic) sequences, the best we can hope for is to predict the true μ -probability of x_t given $x_{<t}$ — not the actual value of x_t itself.

Corollaries 3.3.1 and 3.3.2: Convergence of ξ to μ

Corollary 3.3.1 (Convergence of KL divergence to zero)

The KL divergence satisfies $d_t(x_{<t}) \rightarrow 0$ with μ -probability 1.

Plain English. Since $D_\infty = \sum_t \mathbf{E}_\mu[d_t] \leq \ln w_\mu^{-1} < \infty$ (Theorem 3.2.5), the terms d_t must shrink to zero almost surely along the way — an infinite sum of non-negative terms can only be finite if the terms themselves vanish (in an appropriate averaged sense).

Corollary 3.3.2 (Convergence of ξ to μ)

The off-sequence difference (and the on-sequence ratio) between ξ and μ goes to zero (one), in the sense that

$$\lim_{t \rightarrow \infty} |\xi(x'_t | x_{<t}) - \mu(x'_t | x_{<t})| = 0 \quad \text{and} \quad \lim_{t \rightarrow \infty} \frac{\xi(x_t | x_{<t})}{\mu(x_t | x_{<t})} = 1$$

with μ -probability 1 and in mean² sum, for any sequence x'_t .

Plain English. “Off-sequence” means this holds for *any* candidate next symbol x'_t , not just the one that actually occurs — a strong and useful property. The ratio convergence, however, only holds “on-sequence,” i.e. evaluated at the symbol x_t that was actually sampled from μ .

Corollary 3.3.3 and Theorem 3.3.4: Counting the Errors

Corollary 3.3.3 (Expected number of ε -errors between ξ and μ)

For any $\varepsilon > 0$, the μ -expected number of times that $d_t(x_{<t})$ exceeds ε (epsilon) is

$$\mathbf{E}_{\mu} \left[\sum_{t=1}^{\infty} \mathbb{I}[d_t > \varepsilon] \right] \leq \varepsilon^{-1} \ln w_{\mu}^{-1} < \infty$$

Plain English. $\mathbb{I}[\cdot]$ is the indicator function (1 if true, 0 if false). This says the number of time steps where ξ 's prediction is “badly wrong” (KL distance above threshold ε) is, on average, finite — bad predictions become rarer and rarer and eventually stop happening altogether.

Theorem 3.3.4 (High-probability bound on ξ from μ deviation)

For any $\delta > 0$ (delta), with μ -probability at least $1 - \delta$,

$$\sum_{t=1}^{\infty} h_t \leq \ln w_{\mu}^{-1} + 2 \ln \frac{1}{\delta} \quad \text{and} \quad \sum_{t=1}^{\infty} d_t \leq e \cdot \left(\ln \frac{6}{\delta} \right) \cdot \left(\ln w_{\mu}^{-1} + \ln \frac{2}{\delta} \right)$$

Plain English. These are *high-probability* (not just in-expectation) guarantees: with confidence $1 - \delta$ (e.g. 99%), the *entire infinite sum* of errors stays below an explicit, finite number.

What If $\mu \notin \mathcal{M}$?

All bounds so far assumed the true environment μ actually belongs to our class $\mathcal{M}$. What if it does not — the case of *model misspecification*? We can weaken the assumption: it suffices that some $\hat{\mu} \in \mathcal{M}$ (mu-hat) is merely “close” to μ in KL divergence.

Theorem 3.4.1 (Bound on KL divergence for out-of-class distributions)

Let μ be an arbitrary measure (that may or may not be in $\mathcal{M}$). For $\rho \in \mathcal{M}$, let $\text{KL}_n(\mu\|\rho) := \sum_{x_{1:n}} \mu(x_{1:n}) \ln \frac{\mu(x_{1:n})}{\rho(x_{1:n})}$ denote the KL divergence between ρ and μ restricted to $x_{1:n}$. Then for all n ,

$$D_n \leq \inf_{\rho \in \mathcal{M}} \left\{ \ln w_\rho^{-1} + \text{KL}_n(\mu\|\rho) \right\}$$

Plain English. We can pick *any* ρ in our class as a “proxy” for μ ; the bound then costs us the usual $\ln(1/w_\rho)$ term *plus* however far μ actually is (in KL) from that proxy ρ . Taking the infimum over all choices of $\rho \in \mathcal{M}$ gives the tightest such bound.

Proof of Theorem 3.4.1

Proof. Let $\rho \in \mathcal{M}$ be arbitrary. Using the same technique as in Theorem 3.2.5's proof,

$$\begin{aligned} D_n &= \sum_{x_{1:n}} \mu(x_{1:n}) \ln \frac{\mu(x_{1:n})}{\xi(x_{1:n})} = \sum_{x_{1:n}} \mu(x_{1:n}) \ln \frac{\mu(x_{1:n})}{\rho(x_{1:n})} \frac{\rho(x_{1:n})}{\xi(x_{1:n})} \\ &= \text{KL}_n(\mu \parallel \rho) + \sum_{x_{1:n}} \mu(x_{1:n}) \ln \frac{\rho(x_{1:n})}{\xi(x_{1:n})} \leq \text{KL}_n(\mu \parallel \rho) + \ln w_\rho^{-1} \end{aligned}$$

using $\rho(x_{1:n})/\xi(x_{1:n}) \leq w_\rho^{-1}$ (Proposition 3.1.4). Since this holds for every $\rho \in \mathcal{M}$, it holds for the minimizing one. ■

Corollaries 3.4.2 and 3.4.3

Corollary 3.4.2 (Out-of-class KL bound)

If $\hat{\mu} \in \mathcal{M}$ satisfies $\text{KL}_{\infty}(\mu \parallel \hat{\mu}) < \infty$, then $D_{\infty} < \infty$, hence ξ converges to μ .

Corollary 3.4.3 (Average KL limits to zero)

Let ρ be an environment satisfying $\text{KL}_n(\mu \parallel \rho) = o(n)$ (grows sub-linearly). Then the average KL divergence D_n/n converges to zero.

Plain English. “ $o(n)$ ” (little-o) means “grows strictly slower than n .” Dividing Theorem 3.4.1 by n : even if the KL cost never vanishes entirely, as long as it grows slower than linearly, its *average* contribution washes out as $n \rightarrow \infty$ (a weaker, Cesàro-style notion of convergence).

Definition 3.5.1: The Predictor Model

We now use predictions to make *decisions*, incurring a *loss* depending on the action taken and the outcome observed. We restrict to *passive environments*, where the environment's behavior does not depend on the actions taken (e.g. weather forecasting, or trading stocks with too little capital to move the market).

Definition 3.5.1 (Predictor model)

A *predictor model* is comprised of a finite set of *observations* $\mathcal{X}$ and *actions* $\mathcal{Y}$, a probability semimeasure $\mu : \mathcal{X}^* \rightarrow \Delta\mathcal{X}$ called the *environment*, a predictor $\Lambda : \mathcal{X}^* \rightarrow \mathcal{Y}$ (capital lambda), and a loss function $loss : \mathcal{X} \times \mathcal{Y} \rightarrow [0, 1]$. On each time step t , given history $x_{<t}$, an observation $x_t \sim \mu(\cdot | x_{<t})$ is sampled from the environment. The predictor Λ produces action y_t^Λ (based solely on $x_{<t}$) and suffers loss $loss(x_t, y_t^\Lambda)$. The time step t is incremented, and the interaction loops.

Plain English. At every round: nature reveals an observation x_t ; *before* seeing it, the predictor already committed to an action y_t (using only the past); a loss between 0 and 1 is charged based on how well that action matches what actually happened.

Example 3.5.3: Weather Prediction

Example 3.5.3 (Weather prediction)

The predictor can choose to wear or not wear a jacket. If it is cold, the jacket is a great help; if it is warm, the jacket is a mild annoyance to carry. Actions $\mathcal{Y} = \{\text{jacket, no jacket}\}$, observations $\mathcal{X} = \{\text{warm, cold}\}$, with loss matrix:

<i>loss</i>	Warm	Cold
Jacket	$l_{wj} := 0.3$	$l_{cj} := 0.1$
No Jacket	$l_{wn} := 0.0$	$l_{cn} := 1.0$

Plain English. Wearing a jacket when it is warm is a small inconvenience (loss 0.3); leaving the jacket at home when it is cold is disastrous (loss 1.0). The asymmetry in this loss matrix is exactly what will make caution (wearing a jacket) sensible even when warm weather is somewhat more likely than cold weather.

Definitions 3.5.4–3.5.5: Expected Loss and Optimal Predictors

Definition 3.5.4 (ν -expected instantaneous loss)

The ν -expected instantaneous loss when Λ predicts the t -th symbol given history $x_{<t}$ from environment ν is

$$\text{Loss}_t^\nu(\Lambda) := \mathbf{E}_\nu[\text{loss}(\cdot, y_t^\Lambda) | x_{<t}] = \sum_{x_t} \nu(x_t | x_{<t}) \text{loss}(x_t, y_t^\Lambda)$$

We write $\text{Loss}_t(\Lambda) := \text{Loss}_t^\mu(\Lambda)$ by default (true environment).

Definition 3.5.5 (ρ -optimal predictor Λ_ρ)

Given a semimeasure ρ , Λ_ρ is the predictor that minimizes ρ -expected loss:

$$y_t^{\Lambda_\rho} := \arg \min_{y_t \in \mathcal{Y}} \sum_{x_t} \rho(x_t | x_{<t}) \text{loss}(x_t, y_t).$$

Theorem 3.5.6: Λ_ν Minimizes ν -Expected Loss

Theorem 3.5.6 (Λ_ν minimizes ν -expected loss)

Let Λ be any predictor. Then Λ_ν achieves minimal expected loss in environment ν : for all t , $loss_t^\nu(\Lambda_\nu) \leq loss_t^\nu(\Lambda)$.

Plain English. If a predictor happens to know the true environment exactly, greedily picking the action that minimizes *expected* loss under that environment is provably optimal — this is just decision theory 101, but it sets the baseline every other predictor is compared to.

Proof. By definition, $Loss_t^\nu(\Lambda) = \sum_{x_t} \nu(x_t|x_{<t}) loss(x_t, y_t^\Lambda)$ is minimized by choosing $y_t^\Lambda = \arg \min_{y_t \in \mathcal{Y}} \sum_{x_t} \nu(x_t|x_{<t}) loss(x_t, y_t)$, which is precisely what Λ_ν does. ■

Example 3.5.7: Shall Bayes Take His Jacket? (I)

Assume the weather is (unrealistically) i.i.d. with $\theta := \mathbf{P}_\theta(x_t = \text{warm})$. The θ -expected losses for (not) taking a jacket are

$$l_j := \theta l_{wj} + (1 - \theta) l_{cj}, \quad l_n := \theta l_{wn} + (1 - \theta) l_{cn}$$

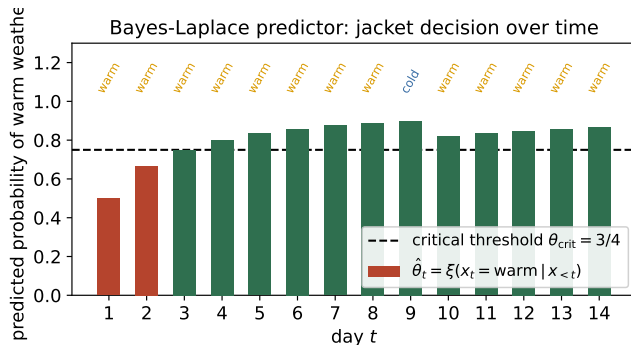
Taking a jacket minimizes loss iff $\theta < \theta_{\text{critical}} := \frac{l_{cn} - l_{cj}}{l_{cn} - l_{cj} + l_{wj} - l_{wn}} = \frac{3}{4}$. If we *knew* $\theta_{\text{true}} = 0.9$, predictor Λ_μ picks **no jacket**. For *unknown* μ : let $\mathcal{M}$ contain all i.i.d. ν_θ , $\theta \in [0, 1]$, uniform prior density $w(\theta) = 1$ (the Bayes–Laplace model of Section 2.4). The mixture predicts “warm” with probability

$$\hat{\theta}_t := \frac{\# \text{warm} + 1}{t + 1} = \xi(x_t = \text{warm} | x_{<t}), \quad y_t^{\Lambda_\xi} = \text{jacket} \iff \hat{\theta}_t < \frac{3}{4}$$

Plain English. #warm counts warm days seen so far among $x_{<t}$, and $\theta_{\text{critical}} = 3/4$ is the break-even chance of warm weather at which taking a jacket stops being worthwhile, given the asymmetric loss matrix of Example 3.5.3.

Example 3.5.7: Shall Bayes Take His Jacket? (II)

Plain English. The Reverend Thomas Bayes starts ($t=1$) believing $\hat{\theta}_1 = \frac{1}{2}$ by symmetry, so his loss-minimizing choice is to bring a jacket. After three consecutive warm days he finally forgoes it, since $\hat{\theta}_4 = \frac{4}{5} > \frac{3}{4}$.



green bars = "no jacket" decision, red bars = "take jacket" decision

Python: Implementing Λ_ξ for the Weather Example

A direct implementation of the Bayes-optimal predictor Λ_ξ (Definition 3.5.5) using the Laplace estimate $\hat{\theta}_t$, matching Example 3.5.7 exactly (Exercise 8 asks for precisely this).

```
l_wj, l_cj, l_wn, l_cn = 0.3, 0.1, 0.0, 1.0
theta_crit = (l_cn - l_cj) / (l_cn - l_cj + l_wj - l_wn)    # = 0.75

def bayes_predictor(history):                                # history: list of 'warm'/'cold'
    n_warm = history.count('warm')
    t = len(history)
    theta_hat = (n_warm + 1) / (t + 1)                       # xi(x_t=warm | x_{<t})
    return ('no jacket' if theta_hat >= theta_crit else 'jacket'), theta_hat

history = []
for day, weather in enumerate(['warm', 'warm', 'warm', 'warm'], start=1):
    action, theta_hat = bayes_predictor(history)
    print(f"day {day}: hat_theta={theta_hat:.3f} -> {action}")
    history.append(weather)

# day 1: hat_theta=0.500 -> jacket
# day 2: hat_theta=0.667 -> jacket
# day 3: hat_theta=0.750 -> no jacket    (exactly at the threshold)
# day 4: hat_theta=0.800 -> no jacket
```

Plain English. Notice how Λ_ξ keeps the jacket cautiously even though $\hat{\theta}_2 = 0.667$ already predicts warm weather is more likely than not — because the asymmetric loss matrix makes “cold without a jacket” far worse than “warm with a jacket.”

Theorem 3.5.8: Bounding the Loss Gap by KL Divergence

Theorem 3.5.8 (Bounding loss difference by KL divergence)

$$0 \leq \text{Loss}_t(\Lambda_\xi) - \text{Loss}_t(\Lambda_\mu) \leq a_t(x_{<t}) \leq \sqrt{2 d_t(x_{<t})}$$

Plain English. Λ_μ is the ideal predictor that knows the true environment; Λ_ξ only has access to the Bayes mixture. This theorem says the *extra* loss Λ_ξ suffers, relative to the unbeatable Λ_μ , at any single time step, is controlled by how far ξ still is from μ (via the absolute distance a_t , itself bounded by $\sqrt{2d_t}$, Theorem 3.2.3). Since $d_t \rightarrow 0$ w.p.1 (Corollary 3.3.1), this gap vanishes with μ -probability 1 — but says nothing yet about *how fast*.

Proof idea. Expand the difference, add and subtract the cross-terms $\text{Loss}_t^\xi(\Lambda_\mu) - \text{Loss}_t^\xi(\Lambda_\xi) \geq 0$ (Theorem 3.5.6 with $\nu = \xi$), and bound the remaining sum by

$$\sum_{x_t} |\mu(x_t|x_{<t}) - \xi(x_t|x_{<t})| \cdot |\text{loss}(x_t, y_t^{\Lambda_\xi}) - \text{loss}(x_t, y_t^{\Lambda_\mu})| \leq a_t(x_{<t}), \text{ since losses lie in } [0, 1].$$

Definition 3.5.9 and Example 3.5.10: Total Loss

Definition 3.5.9 (Total loss)

The *total loss* of a prediction scheme Λ_ρ over time steps 1 to n is defined as
$$\text{Loss}_{1:n}(\Lambda_\rho) := \sum_{t=1}^n \mathbf{E}_\mu[\text{Loss}_t(\Lambda_\rho)].$$

Plain English. Unlike the per-step loss (bounded by 1), the total loss accumulated by Λ_μ over n steps may itself be *unbounded*, growing linearly in n .

Example 3.5.10 (Linearly growing loss in stochastic environments)

An unfair coin with $\theta = 0.7$ probability of heads. The predictor guesses the outcome, loss 0 (resp. 1) for a correct (resp. incorrect) guess. Even knowing the true environment, the optimal predictor always guesses heads and still incurs expected loss 0.3 every single time step — so over infinitely many flips, the total loss *diverges*.

Plain English. There is an unavoidable, irreducible loss coming from the environment's own randomness — Λ_μ 's loss can be unbounded even though it is optimal. So we should instead bound the excess loss of Λ_ξ over Λ_μ , not the raw total loss itself.

Theorems 3.5.11–3.5.12: Bounding the Excess Loss

Theorem 3.5.11 (Merhav–Feder bound [MF98])

$$0 \leq \text{Loss}_{1:n}(\Lambda_\xi) - \text{Loss}_{1:n}(\Lambda_\mu) \leq A_n \leq \sqrt{2nD_n}$$

Plain English. Summing Theorem 3.5.8 over $t = 1, \dots, n$ gives this bound on the *cumulative* excess loss. The catch: the right-hand side grows like $\sqrt{n}$, so we still cannot rule out the excess loss growing unboundedly (just more slowly than n).

Theorem 3.5.12 (Hellinger loss bound)

$$0 \leq \sqrt{\text{Loss}_{1:n}(\Lambda_\xi)} - \sqrt{\text{Loss}_{1:n}(\Lambda_\mu)} \leq \sqrt{2H_n} \leq \sqrt{2D_n} \leq \sqrt{2 \ln w_\mu^{-1}} < \infty$$

Plain English. A much better bound (proof in Appendix A of [Hut07e], beyond our scope): the *square-root* of total losses differ by at most a constant, independent of n ! The price is that the bound now involves $\sqrt{\text{Loss}}$ rather than Loss directly, which we untangle next.

Corollary 3.5.13 and Corollary 3.5.14

Corollary 3.5.13 (Bounding loss difference between Λ_ξ and Λ_μ)

$$0 \leq \text{Loss}_{1:n}(\Lambda_\xi) - \text{Loss}_{1:n}(\Lambda_\mu) \leq 2H_n + 2\sqrt{2H_n \text{Loss}_{1:n}(\Lambda_\mu)}$$

where we can further bound $H_n \leq D_n \leq \ln w_\mu^{-1} < \infty$ as above.

Plain English. Squaring Theorem 3.5.12 and rearranging recovers a bound directly on $\text{Loss}_{1:n}(\Lambda_\xi) - \text{Loss}_{1:n}(\Lambda_\mu)$ itself: the excess loss grows at most like $O(\sqrt{\text{Loss}_{1:n}(\Lambda_\mu)})$ — the “regret” of not knowing μ grows only as the square root of the unavoidable loss itself.

Corollary 3.5.14 (Loss bounds for infinite sequences)

- (i) $\text{Loss}_{1:\infty}(\Lambda_\xi)$ is finite iff $\text{Loss}_{1:\infty}(\Lambda_\mu)$ is finite.
- (ii) $\text{Loss}_{1:\infty}(\Lambda_\xi) \leq 2D_\infty \leq 2 \ln w_\mu^{-1}$ for deterministic μ and if $\forall x \exists y : \text{loss}(x, y) = 0$.
- (iii) $\frac{\text{Loss}_{1:n}(\Lambda_\mu)}{\text{Loss}_{1:n}(\Lambda_\xi)} \rightarrow 1$ whenever $\text{Loss}_{1:n}(\Lambda_\mu) \xrightarrow{n \rightarrow \infty} \infty$.

Plain English. All of these hold regardless of the choice of loss function or true environment μ , as long as $\mu \in \mathcal{M}$: the relative penalty of using ξ instead of the unattainable Λ_μ vanishes in the long run.

Definition 3.6.1: Pareto Optimality

We have seen Λ_ξ incurs excess loss growing like $O(\sqrt{\text{Loss}_{1:n}(\Lambda_\mu)})$ relative to the best-informed predictor. But is Λ_ξ the *best possible* predictor that does not know μ ? We can ask: is there a *better* predictor Λ that beats Λ_ξ on some environment without doing worse on another?

Definition 3.6.1 (Pareto Optimality of ξ)

Let $\mathcal{F}(\mu, \rho)$ be any performance measure of ρ relative to μ . A predictor ρ is called *Pareto optimal* in class $\mathcal{M}$ with respect to $\mathcal{F}$ if there is no ρ' such that $\mathcal{F}(\nu, \rho') \leq \mathcal{F}(\nu, \rho)$ for all $\nu \in \mathcal{M}$, and $\mathcal{F}(\nu, \rho') < \mathcal{F}(\nu, \rho)$ for at least one $\nu \in \mathcal{M}$.

Plain English. A predictor is Pareto optimal if it cannot be *strictly* improved on any single environment without becoming worse on some other environment — there is no “free lunch” improvement available. It turns out ξ is Pareto optimal, with respect to *any* loss function, and with respect to the KL divergence too.

Theorem 3.6.2: Pareto Optimality w.r.t. Loss

Theorem 3.6.2 (Pareto Optimality of Λ_ξ w.r.t. Loss)

The Bayes-optimal predictor Λ_ξ is Pareto optimal with respect to $\mathcal{F}(\nu, \Lambda) = \text{Loss}_{1:n}^\nu(\Lambda)$.

Proof (by contradiction). Suppose not: there exists Λ with $\text{Loss}_{1:n}^\nu(\Lambda) \leq \text{Loss}_{1:n}^\nu(\Lambda_\xi)$ for all $\nu \in \mathcal{M}$, and strict inequality for some ν . Using Definition 3.1.2,

$$\text{Loss}_{1:n}^\xi(\Lambda) = \sum_{\nu} w_{\nu} \text{Loss}_{1:n}^{\nu}(\Lambda) < \sum_{\nu} w_{\nu} \text{Loss}_{1:n}^{\nu}(\Lambda_{\xi}) = \text{Loss}_{1:n}^{\xi}(\Lambda_{\xi}) \leq \text{Loss}_{1:n}^{\xi}(\Lambda)$$

where the last step is Theorem 3.5.6 with $\nu = \xi$. This gives $\text{Loss}_{1:n}^{\xi}(\Lambda) < \text{Loss}_{1:n}^{\xi}(\Lambda)$, a contradiction. ■

Plain English. If a rival predictor Λ really did dominate Λ_ξ on every environment (and strictly on one), it would also have to beat Λ_ξ under the mixture ξ itself, on average — but Λ_ξ is *defined* to be the best possible predictor under ξ (Theorem 3.5.6), so no such Λ can exist.

Theorem 3.6.3: Pareto Optimality w.r.t. KL Divergence

Theorem 3.6.3 (Pareto Optimality of ξ w.r.t. KL divergence)

The mixture ξ is Pareto optimal with respect to the KL divergence $\text{KL}_{1:n}(\nu\|\rho) := \sum_{\mathbf{x}_{1:n}} \nu(\mathbf{x}_{1:n}) \ln \frac{\nu(\mathbf{x}_{1:n})}{\rho(\mathbf{x}_{1:n})}$.

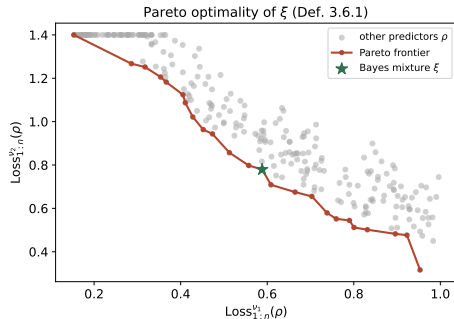
Proof sketch. Assume some ρ dominates ξ on every ν (with strict improvement on one). Summing over $\mathcal{M}$ weighted by w_ν gives $\text{KL}_{1:n}(\xi\|\rho) < 0$, contradicting the non-negativity of KL divergence (Corollary 2.5.16). ■ A similar result holds for the instantaneous distances of Definition 3.2.1 under mild extra assumptions (Exercise 14).

Definition 3.6.4: Balanced Pareto Optimality

Definition 3.6.4 (Balanced Pareto Optimality)

ξ is called *balanced Pareto optimal* with respect to $\mathcal{F}$ and weights $\{w_\nu\}_{\nu \in \mathcal{M}}$ if for every ρ , $\sum_{\nu \in \mathcal{M}} w_\nu (\mathcal{F}(\nu, \rho) - \mathcal{F}(\nu, \xi)) \geq 0$.

Plain English. Plain Pareto optimality is a fairly weak, minimal requirement. *Balanced* Pareto optimality is stronger: it says that if a rival ρ improves on ξ for some environments, the (prior weighted) sum of *losses* elsewhere must more than compensate — ξ also satisfies this stronger property.



3.7.1 Choices for Model Class $\mathcal{M}$

So far $\mathcal{M}$ and w_ν were left generic. Now we discuss *concrete* universal choices suitable for AGSI (Artificial General Super-Intelligence).

Argument for $\mathcal{M}_{comp}$. [Sol64] suggested the class of all *computable* probability measures: incomputable ν are impractical, while (by the Church–Turing thesis) all known physical processes are stochastically computable, so $\mu \in \mathcal{M}_{comp}$ is a reasonable assumption — and $\mathcal{M}_{comp}$ is the largest practically relevant class.

Argument for $\mathcal{M}_{sol}$. A drawback of $\mathcal{M}_{comp}$: the resulting mixture ξ fails to be computable [Hut05b]. Choosing the larger class $\mathcal{M}_{sol} \supset \mathcal{M}_{comp}$ of all *lower semicomputable* semimeasures makes ξ itself lower semicomputable [Hut05b, LV19]! The reason: lower semicomputable semimeasures can be effectively *enumerated*, but there is no computable enumeration of computable measures (due to the Halting problem [ZL70b, LV19]).

Smaller choices. For practical purposes we often need much smaller classes: MDPs (Markov Decision Processes, Algorithm 11.1), variable-order Markov / Context Tree Weighting (Algorithm 12.4), or the historically first and simplest class, Bernoulli(θ) (Section 2.4.3, Example 3.5.7).

Definition 3.7.1: $\mathcal{M}_{comp}$ and $\mathcal{M}_{sol}$

Definition 3.7.1 ($\mathcal{M}_{comp}$ and $\mathcal{M}_{sol}$)

- $\mathcal{M}_{comp}$ is the set of all *computable* measures over infinite sequences $\mathcal{X}^\infty$.
- $\mathcal{M}_{sol}$ is the set of all *lower semicomputable* semimeasures (see Definitions 2.2.11 and 2.6.12).

Plain English. “Computable” means an algorithm can, in principle, calculate the exact probability to any desired precision in finite time. “Lower semicomputable” is weaker (recall Section 2.6.3): an algorithm can produce a sequence of ever-improving *lower bounds* that converge to the true value, without ever certifying how close the current estimate is. $\mathcal{M}_{sol}$ is chosen mainly for the technical reason that it is *enumerable* (we can effectively list all its members one by one), even though $\mathcal{M}_{comp}$ is conceptually the more natural class.

3.7.2 Choices for Prior w_ν : Three Philosophies

As long as $w_\mu > 0$, ξ is (balanced) Pareto optimal regardless of the *particular* choice of weights w_ν . Still, there are philosophical arguments for particular choices.

- **Principle of Indifference (Pol).** Assign all environments the same weight, $w_{\nu_1} = w_{\nu_2}$. Sensible for *finite* $\mathcal{M}$, but breaks down for infinite (even countable) $\mathcal{M}$: any valid prior must have $\sum_\nu w_\nu \leq 1$, forcing weights to eventually vanish.
- **Maximum entropy principle.** Weight ν by its own entropy — flatter distributions get higher weight, since they encode the fewest extra assumptions. Same drawback as Pol over infinite classes.
- **Simplicity principle.** Weight *simpler* environments more heavily, formalized via *Kolmogorov complexity* K (Section 2.7.2).

Plain English. The first two approaches are natural generalizations of “treat everything symmetrically,” but they simply cannot work once $\mathcal{M}$ is infinite. This motivates the simplicity principle instead.

Occam's Razor, Epicurus' Principle, and the Emerald Example

- **Occam's razor:** “Entities should not be multiplied beyond necessity.” Of all theories consistent with the data, choose the *simplest*.
- **Epicurus' principle:** If more than one theory is consistent with the observations, keep *all* of them (don't rule anything out prematurely).

Combining both. Assign *non-uniform* weights, favoring simpler hypotheses, but never assigning zero weight to any hypothesis not yet ruled out.

Example (Emeralds)

$\mathcal{M} = \{\nu_1, \nu_2\}$, where in ν_1 all emeralds are green forever, and in ν_2 all emeralds are green until the year 2050, after which they turn blue. Both are equally well supported by every observation made before 2050 — but Occam says ν_1 (the simpler rule) should get more weight, while Epicurus insists $w_{\nu_2} \neq 0$: however unlikely, we should not rule it out until it is actually contradicted by evidence.

A “Proof” of Occam’s Razor

Argument. For a countable outcome space $\mathbb{N} (= \{1, 2, 3, \dots\})$, *any* probability distribution $P = \{p_1, p_2, \dots\}$ over $\mathbb{N}$ must have a convergent sum $\sum_i p_i = 1 < \infty$. This forces the tail sum $\sum_{i \geq n} p_i \rightarrow 0$ as $n \rightarrow \infty$: the probability mass p_i assigned to large index i *must* vanish. **The axioms of probability theory alone enforce a simplicity bias**, once the index i is interpreted as “complexity rank.”

Applying this to a countable model class $\mathcal{M} = \{\nu_1, \nu_2, \dots\}$: *any* prior w on $\mathcal{M}$ necessarily assigns vanishing probability to environments with large index i . And any effective enumeration of $\mathcal{M}_{sol}$ also has this property with respect to Kolmogorov complexity: for any k , there exists an index i_k such that $K(\nu_i) > k$ for all $i > i_k$. **So any prior on $\mathcal{M}_{sol}$ necessarily incorporates Occam’s razor** to some degree, purely as a mathematical consequence of being a valid probability distribution over a countably infinite set.

Definition 3.7.2: The Solomonoff Prior

Definition 3.7.2 (Solomonoff prior)

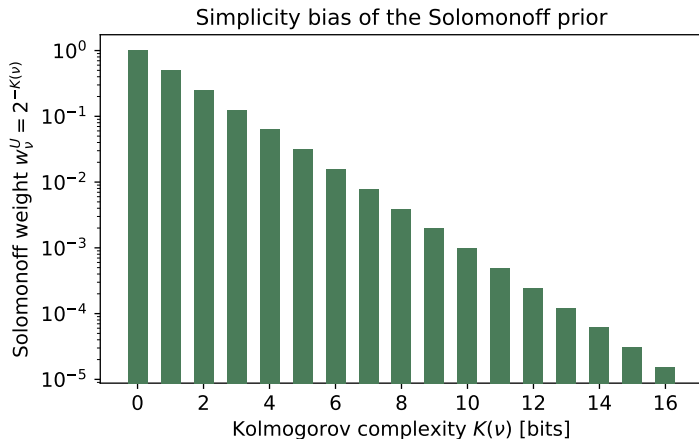
Given a (semi)computable (semi)measure $\nu \in \mathcal{M}_{sol}$, the *Solomonoff prior* assigns a-priori weight

$$w_{\nu}^U := 2^{-K(\nu)}$$

to it. We let ξ_U denote the corresponding Bayesian mixture over $\mathcal{M}_{sol}$ using the weights $w_{\nu} = w_{\nu}^U$.

Plain English. $K(\nu)$ is the Kolmogorov complexity of ν (Section 2.7.2): the length, in bits, of the shortest program that describes ν . Environments with a *short* description get an *exponentially larger* weight than those needing a long description — a precise, quantitative version of Occam's razor. As long as the reference machine used to define K is universal, $2^{-K(\nu)} > 0$ for every $\nu \in \mathcal{M}_{sol}$, satisfying Epicurus too.

The Solomonoff Prior, Visualized



Plain English. Each extra bit of Kolmogorov complexity $K(\nu)$ halves the prior weight w_ν^U — an exponential simplicity bias, shown here on a logarithmic y-axis where the decay appears as a straight line.

Theorem 3.7.3: The Solomonoff Bound

Using the MDL bound (Theorem 2.7.23), $K(\nu) \stackrel{+}{\leq} \log_2[w_\nu'^{-1}]$ for any other lower-semicomputable prior w' : bounds for ξ_U depending on $\ln(w_\nu^U)^{-1}$ are, up to an additive constant, at least as tight as bounds for *any other* choice of prior.

Theorem 3.7.3 (Solomonoff bound)

For sequences sampled from any computable measure $\mu \in \mathcal{M}_{comp}$, the total expected squared error S_∞^U (Definition 3.2.2) of the lower semicomputable Solomonoff predictor $\xi_U \in \mathcal{M}_{sol}$ is bounded as

$$S_\infty^U \leq \ln [(w_\mu^U)^{-1}] = K(\mu) \cdot \ln 2$$

Plain English. Applying our Generalized Solomonoff Bound (Theorem 3.2.5) to this specific choice of prior: the total prediction error is bounded by the Kolmogorov complexity of the *true* environment μ , measured in bits (times $\ln 2$ to convert to nats). Among all possible lower semicomputable priors, this is — up to an additive constant — the *tightest* possible such bound.

Python: Simplicity Bias of the Solomonoff Prior

A toy illustration: suppose four candidate environments have (hypothetical) Kolmogorov complexities $K = 3, 7, 7, 12$ bits. The Solomonoff weights $w_\nu^U = 2^{-K(\nu)}$ decay exponentially with complexity.

```
models = {'nu_simple(K=3)': 3, 'nu_a(K=7)': 7, 'nu_b(K=7)': 7, 'nu_complex(K=12)': 12}

raw_w = {name: 2.0**(-K) for name, K in models.items()}
total = sum(raw_w.values())
w_U = {name: w / total for name, w in raw_w.items()}    # renormalize (sum<=1 already)

for name, K in models.items():
    print(f"{name:18s}  K={K:2d}  w_U={raw_w[name]:.6f}  (normalized: {w_U[name]:.4f})")
# nu_simple(K=3)      K= 3  w_U=0.125000  (normalized: 0.6234)
# nu_a(K=7)           K= 7  w_U=0.007812  (normalized: 0.0390)
# nu_b(K=7)           K= 7  w_U=0.007812  (normalized: 0.0390)
# nu_complex(K=12)    K=12  w_U=0.000244  (normalized: 0.0012)
```

Plain English. A model that is just 4 bits more complex than another receives roughly $2^4 = 16\times$ less prior weight — an exponential penalty on description length that is the mathematical heart of the simplicity principle.

Why a Second Predictor? Motivation for M

ξ_U is a powerful Pareto-optimal predictor with strong bounds, but relies on heavy machinery: semimeasures, a computable enumeration of them, priors, Bayes mixtures, and Kolmogorov complexity. We now introduce the *Solomonoff distribution* M , which has a much simpler definition, relying only on the existence of a **universal Turing machine** (Definition 2.7.2). We restrict to the **binary alphabet** $\mathbb{B}$ in this section, as is customary in algorithmic information theory (results generalize to $|\mathcal{X}|$ -ary alphabets by replacing powers of 2 with powers of $|\mathcal{X}|$).

Remarkably, we will show M is (up to a constant, and for a suitable choice of machine, *exactly*) equal to ξ_U — so M also learns to predict any unknown computable stochastic environment μ , as well as a predictor with direct knowledge of μ .

3.8.1: M from a Uniform Prior over Programs I

Assume for now μ is deterministic and computable: the unique string $x_{1:\infty}$ it generates satisfies $\mu(x_{1:t}) = 1$ for all t , and is produced by some program p_μ of length $\leq l$ bits on a universal monotone Turing machine U . Consider the set of all deterministic programs p on U with $\ell(p) \leq l$ that produce a string with prefix $x \equiv x_{1:t} \sqsubset x_{1:\infty}$. Padding all such programs to length exactly l (using a modified machine U' without the prefix condition), let

$$N_l(x) := |\{p \in \mathbb{B}^l : x \sqsubseteq U'(p)\}|$$

be the number of length- l programs producing strings prefixed by x . By Epicurus' principle, we assume a priori all length- l programs are equally likely, giving probability $P_l(x) := N_l(x)/2^l$ to the string x .

3.8.1: M from a Uniform Prior over Programs II

Semimeasure property. Partitioning programs by their behavior right after outputting x (halt, output a further 0, or output a further 1):

$$N_I(x) \geq |\{p : x0 \sqsubseteq U'(p)\}| + |\{p : x1 \sqsubseteq U'(p)\}| = N_I(x0) + N_I(x1)$$

so $P_I(x) \geq P_I(x0) + P_I(x1)$: P_I is a (sub-probability) semimeasure. **Monotonicity in I .** One can show $P_{I+1}(x) \geq P_I(x)$: extending the program length by one bit can only ever add more programs producing x , never remove any. **Plain English.** As I grows, we consider longer and longer programs, so any given string x (however it is generated) eventually gets a nonzero, non-decreasing chance of appearing.

3.8.1: M from a Uniform Prior over Programs III

Since $P_1(x), P_2(x), \dots$ is monotonically increasing and bounded by 1, it has a well-defined limit as $l \rightarrow \infty$. We define

$$M(x) := \lim_{l \rightarrow \infty} P_l(x) \quad (3.8.1)$$

Plain English. This first construction “over-counts”: every extension of the same underlying program is counted separately at each length l . An equivalent, cleaner construction *weights by program length directly*, counting each *minimal* program exactly once, which we adopt as our official definition next.

Definition 3.8.2: The Solomonoff Distribution

Definition 3.8.2 (Solomonoff distribution)

Given a universal monotone Turing machine U , the *Solomonoff distribution* or *universal distribution* is defined as

$$M_U(x) := \sum_{p: U(p)=x*} 2^{-\ell(p)} \quad (3.8.3)$$

The *conditional Solomonoff distribution* is defined the usual way: $M_U(x|y) := M_U(yx)/M_U(y)$ (3.8.4). When U is the reference UTM, we simply write $M(\cdot) := M_U(\cdot)$.

Plain English. The notation “ $U(p) = x*$ ” means: running program p on U outputs a string that *starts with* x (possibly more after). $M_U(x)$ sums $2^{-\ell(p)}$ (a weight that shrinks exponentially with program length $\ell(p)$) over every *minimal* program that produces a string beginning with x .

Piping uniform noise through a monotone UTM. Equivalently, $M(x)$ is the probability that U , fed uniform random noise on its input tape, outputs a string starting with x : since a program p is “generated” by random noise with probability $2^{-\ell(p)}$ (each of its bits matches with probability $\frac{1}{2}$), summing over all such p recovers exactly (3.8.3). This is the most intuitive picture of M : *throw random bits at a universal computer and see what comes out.*

M vs. No Free Lunch, and Implicit Stochastic Coverage

Comparison to No Free Lunch. The No Free Lunch (NFL) theorems [WM97] also feed a uniform distribution into an algorithm, to argue that averaged over *all* possible problems, no algorithm beats any other. Crucially, NFL requires all prediction problems to be equally likely — which is *not* the setting here: feeding random noise into a *universal Turing machine* (not into “problem space” directly) biases the output strings toward *simplicity*, since simple strings have many short describing programs. Real-world prediction problems are overwhelmingly *structured*, not drawn uniformly from the space of all possible problems — exactly what M exploits.

M implicitly contains all stochastic environments too. The construction above assumed μ deterministic. But every stochastic semimeasure can be written as a convex (weighted) combination of deterministic semimeasures, and M is itself (as we will see) a mixture over deterministic semimeasures — so M implicitly contains, and learns to predict, stochastic environments too, for free.

3.8.2 Properties: M is a Semimeasure, Lower Semicomputable I

Theorem 3.8.5 (M is a semimeasure)

From the first representation of M (3.8.1), $M(\epsilon) = 1$, and for every x , $M(x) > M(x0) + M(x1)$ (strict inequality).

Proof. $M(\epsilon) = \lim_I N_I(\epsilon)/2^I = \lim_I |\{p \in \mathbb{B}^I : \epsilon \sqsubseteq U'(p)\}|/2^I = 1$ (every program has the empty prefix). Writing $U(p) = x\perp$ to mean U halts right after outputting x (or runs forever without further output),

$$M(x) = \sum_{p: U(p)=x*} 2^{-\ell(p)} = \sum_{p: U(p)=x\perp} 2^{-\ell(p)} + \sum_{p: U(p)=x0*} 2^{-\ell(p)} + \sum_{p: U(p)=x1*} 2^{-\ell(p)} > M(x0) + M(x1)$$

strictly, since a program with $U(p) = x\perp$ always exists, contributing a strictly positive extra term. This also shows M is *not* a probability measure (there is always “leftover” probability mass). ■

3.8.2 Properties: M is a Semimeasure, Lower Semicomputable II

Theorem 3.8.6 (M is lower semicomputable)

M can be approximated from below by a computable, monotonically increasing sequence.

Proof idea. Effectively enumerate *all* programs $p_1, p_2, \dots$ for U and dovetail (interleave) the simulation of $U(p_1), U(p_2), \dots$. Whenever a simulated program either reads past its own end or outputs a string of length $\ell(x)$, halt that simulation and check if $U(p_i) = x^*$; if so, add $2^{-\ell(p_i)}$ to a running total $\hat{M}$.

Running this for t steps gives an estimate $\hat{M}$ that grows monotonically with t and, for any $\varepsilon > 0$, exceeds $M(x) - \varepsilon$ for t large enough (since the tail of the defining sum vanishes). This is precisely the definition of lower semicomputable (Definition 2.6.12). ■

Plain English. We can always get a running *lower bound* estimate of $M(x)$ that keeps improving over time, but (since M relies on Kolmogorov complexity, which is itself only upper semicomputable) we can never know exactly how close our current estimate is to the true value.

Theorem 3.8.7: The Kolmogorov–Solomonoff Sandwich

Theorem 3.8.7 (Kolmogorov–Solomonoff sandwich)

$$0 \leq K(x|\ell(x)) \stackrel{+}{\leq} -\log_2 M(x) \leq Km(x) \stackrel{+}{\leq} K(x) \stackrel{+}{\leq} \ell(x) + 2\log_2 \ell(x)$$

Plain English. $\stackrel{+}{\leq}$ means “ $\leq$ up to an additive constant.” $K(x|\ell(x))$ is the Kolmogorov complexity of x given its own length; Km (monotone Kolmogorov complexity) is a close cousin of K restricted to *monotone* programs. This chain of inequalities “sandwiches” $-\log_2 M(x)$ tightly between two well-studied complexity measures: $-\log_2 M(x)$ behaves, up to an additive constant, just like the *descriptive complexity* of x — strings that are algorithmically simple get a *high* probability under M , and vice versa. Only M (not Km or K) has genuinely excellent predictive properties, as we will see next; Km can only predict deterministic sequences with much worse bounds, and K cannot be used for prediction at all.

3.8.3 Equivalence of M and ξ_U

Theorem 3.8.8 (Solomonoff equivalence)

The Solomonoff distribution M is equal to the Bayesian mixture ξ_U up to an irrelevant multiplicative constant,

$$\forall \text{ UTM } U : M(x) \stackrel{\times}{=} \xi_U(x) \quad \text{and} \quad \exists \text{ UTM } U : M(x) = \xi_U(x)$$

Plain English. $\stackrel{\times}{=}$ means “equal up to a multiplicative constant” (which does not affect any of our asymptotic bounds). Remarkably, M — defined purely via program-counting on a Turing machine, with no explicit reference to any model class or prior — turns out to coincide (up to a harmless constant, and for a well-chosen reference machine, *exactly*) with the very different-looking Bayes-mixture-over-all-semicomputable-semimeasures ξ_U . In fact [WSH11] proves an even stronger statement: the *class* of all Solomonoff priors M_U and the *class* of all Bayesian mixtures ξ_w over $\mathcal{M}_{sol}$ are literally *identical* sets of functions. The full proof is beyond the scope of this book; see [ZL70b, Hut05b, LV19].

3.8.4 Theorem 3.8.9: Deterministic Solomonoff Bound

Theorem 3.8.9 (Deterministic Solomonoff bound)

For any sequence $x_{1:\infty}$,

$$\sum_{t=1}^n (1 - M(x_t | x_{<t}))^2 \leq \sum_{t=1}^n |1 - M(x_t | x_{<t})| \leq Km(x_{1:n}) \ln 2$$

Furthermore, if $x_{1:\infty}$ is a computable sequence, then $M(x_t | x_{<t}) \rightarrow 1$.

Plain English. For a *deterministic* sequence (each symbol either happens or doesn't, no genuine randomness), M 's predicted probability of the actually-occurring symbol converges to certainty ($M \rightarrow 1$), and the total “surprise” is bounded by the sequence's monotone Kolmogorov complexity. Unlike Corollary 3.3.2, this needs no probabilistic language at all — it holds for the one true sequence outright.

Corollary 3.8.10: Solomonoff Distribution Domination

Corollary 3.8.10 (Solomonoff distribution domination)

For all lower semicomputable semimeasures ν : $M(x) \stackrel{\times}{\geq} 2^{-K(\nu)} \nu(x)$.

Plain English. This is the M -analogue of Proposition 3.1.4: M dominates every semicomputable semimeasure ν , up to a constant depending only on ν 's complexity. Combined with Theorem 3.8.8's equivalence of M and ξ_U , this domination property is what allows the predictive-convergence machinery of Sections 3.2–3.3 to transfer almost verbatim from ξ to M , as we see next.

Theorem 3.8.11: Predictive Convergence of M

Theorem 3.8.11 (Predictive convergence of M)

Given a computable measure μ from which a sequence $x_{1:\infty}$ is sampled, we have

$$S_{\infty}^M = \sum_{t=1}^{\infty} \sum_{x_{1:t} \in \mathbb{B}^t} \mu(x_{<t}) (M(x_t|x_{<t}) - \mu(x_t|x_{<t}))^2 \stackrel{+}{\leq} K(\mu) \ln 2 < \infty$$

which implies $M(x'_t|x_{<t})$ converges to $\mu(x'_t|x_{<t})$ with μ -probability 1.

Plain English. This is exactly the same style of bound as Theorem 3.7.3 for ξ_U , but now proved directly for M , using Corollary 3.8.10 in place of Proposition 3.1.4. So even for *stochastic* environments μ , feeding random noise into a universal Turing machine gives a predictor M that eventually learns to predict μ correctly — with total expected squared error bounded by μ 's own Kolmogorov complexity.

Definition 3.9.1: (Super)martingales

Sequences $x_{1:\infty}$ sampled from a probability measure μ are also called *stochastic processes*, viewed as random variables $X_1, X_2, X_3, \dots$. A *different* stochastic process $Z_1, Z_2, Z_3, \dots$ is called a *supermartingale* if it is non-increasing in expectation.

Definition 3.9.1 ((Super)martingale)

The stochastic process $Z_1, Z_2, Z_3, \dots$ is said to be a *martingale* with respect to $X_1, X_2, X_3, \dots$ if for all $t \in \mathbb{N}$, the expectation of $|Z_t|$ is finite and the expectation of Z_t given $X_1, \dots, X_{t-1}$ is Z_{t-1} . Formally,

$$\mathbf{E}[|Z_t|] < \infty \quad \text{and} \quad \mathbf{E}[Z_t | X_1, \dots, X_{t-1}] = Z_{t-1} \quad \text{for martingales}$$

If we weaken the “=” to “ $\leq$ ”, then $Z_1, Z_2, Z_3, \dots$ is a *supermartingale*.

Plain English. A martingale is a “fair game”: given everything observed so far, the expected *next* value equals the *current* value — no systematic drift up or down. A supermartingale is a game that, in expectation, is never favorable to the player holding Z : it can only drift downward or stay flat, never upward, on average.

Example 3.9.2: The Martingale ν/μ

Example 3.9.2 (Martingale ν/μ)

The only (super)martingale (w.r.t. $x_t \sim \mu(\cdot|x_{<t})$) we will encounter in this book is $Z_t := \nu(x_{1:t})/\mu(x_{1:t})$, for the true probability measure μ and some (semi)measure ν :

$\mathbf{E}_\mu[|Z_t|] = \sum_{x_{1:t}} \mu(x_{1:t})Z_t = \sum_{x_{1:t}} \nu(x_{1:t}) \leq 1$, and

$$\mathbf{E}_\mu[Z_t|x_{<t}] = \sum_{x_t} \mu(x_t|x_{<t})Z_t = \sum_{x_t} \mu(x_t|x_{<t}) \frac{\nu(x_t|x_{<t})}{\mu(x_t|x_{<t})} Z_{t-1} = Z_{t-1} \sum_{x_t} \nu(x_t|x_{<t}) \leq Z_{t-1}$$

Plain English. Z_t is the “likelihood ratio” of model ν versus the truth μ , evaluated on the actually-observed history. It is a genuine martingale (equality) whenever ν is a proper probability measure (since $\sum_{x_t} \nu(x_t|x_{<t}) = 1$ exactly), and merely a supermartingale (inequality) if ν is only a semimeasure (some probability “leaks away”).

Theorem 3.9.3: Supermartingale Convergence

Theorem 3.9.3 (Supermartingale convergence theorem [Doo53])

Let $Z_1, Z_2, \dots$ be a non-negative supermartingale (w.r.t. process X_t). Then the sequence converges almost surely to a random variable Z_∞ with finite expectation.

Plain English. Remarkably, *no* assumptions at all are needed on the process X_t itself — only non-negativity and bounded expectation of Z_t — yet convergence is guaranteed. This is why martingales are such a versatile, powerful tool. (The proof relies on Doob's “upcrossing lemma” and is beyond our scope; it is also a purely *asymptotic* result, with no guaranteed convergence *rate*.)

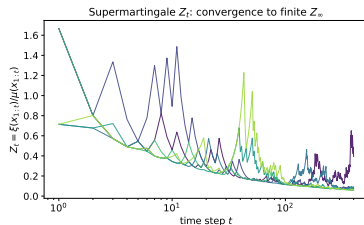
Example 3.9.4: The Martingale ξ/μ

Example 3.9.4 (Martingale ξ/μ)

$Z_t := \xi(x_{1:t})/\mu(x_{1:t})$ is a supermartingale. Theorem 3.9.3 implies $Z_\infty < \infty$ μ -almost surely. From Proposition 3.1.4, $Z_t \geq w_\mu > 0$, hence $Z_\infty > 0$ too. Together,

$$\frac{\xi(x_t|x_{<t})}{\mu(x_t|x_{<t})} = \frac{Z_t}{Z_{t-1}} \xrightarrow{t \rightarrow \infty} \frac{Z_\infty}{Z_\infty} = 1 \quad \text{w.}\mu.\text{p.1}$$

Plain English. This re-derives the on-sequence *ratio* convergence of Corollary 3.3.2 via martingale theory instead of KL-divergence bounds — but it is *weaker*: no convergence *rate* is provided, unlike Corollary 3.3.3 and Theorem 3.3.4.



Python: Simulating the Supermartingale $Z_t = \xi/\mu$

We track $Z_t = \xi(x_{1:t})/\mu(x_{1:t})$ in log-space (avoiding numerical underflow), using the same 99-model mixture as before.

```
import numpy as np
thetas, mu_true = np.arange(1, 100) / 100, 0.7
def simulate_log_Z(seed, n=400):
    rng = np.random.default_rng(seed)
    x = rng.binomial(1, mu_true, size=n)
    log_w, log_Z, running = np.full(99, -np.log(99)), np.zeros(n), 0.0
    for t, xt in enumerate(x):
        w = np.exp(log_w - log_w.max()); w /= w.sum()
        xi1 = np.clip((w * thetas).sum(), 1e-12, 1 - 1e-12)
        p_xi = xi1 if xt == 1 else 1 - xi1
        p_mu = mu_true if xt == 1 else 1 - mu_true
        running += np.log(p_xi) - np.log(p_mu)    # log(xi/mu) increment
        log_Z[t] = running
        log_w += np.where(xt == 1, np.log(thetas), np.log1p(-thetas))
    return np.exp(log_Z)
for seed in range(3):
    Z = simulate_log_Z(seed)
    print(f"seed={seed}: Z_50={Z[49]:.3f}  Z_400={Z[399]:.3f}")
# seed=0: Z_50=1.822  Z_400=1.532  (settles, no persistent drift)
# seed=1: Z_50=0.945  Z_400=1.104
# seed=2: Z_50=2.017  Z_400=1.780
```

Plain English. Each path settles to its own finite limit Z_∞ — the almost-sure convergence of Theorem

Theorem 3.9.5: Merging of Opinions

Theorem 3.9.5 (Merging of opinions [BD62])

$$\sup_A |\mathbf{P}_\xi[A|x_{<t}] - \mathbf{P}_\mu[A|x_{<t}]| \xrightarrow{t \rightarrow \infty} 0 \quad \text{w.}\mu.\text{p.1}$$

where $\sup_A$ ranges over all measurable events $A \subseteq \mathcal{X}^\infty$.

Plain English. This is a much *stronger* statement than Corollary 3.3.2: not only does ξ agree with μ on the probability of the *very next* symbol, but ξ 's and μ 's probabilities agree, *uniformly*, on *every conceivable event* A about the entire future — including events that depend on the infinite tail of the sequence in complicated, non-trivial ways. The proof relies on martingale theory (via Theorem 3.9.3) and the dominance $\mathbf{P}_\xi[A] \geq w_\mu \mathbf{P}_\mu[A]$ for all events A , but is otherwise beyond our scope.

Why this matters. Corollary 3.3.2 only concerns the event $A_t = \{x_{<t}x'_t\} \times \mathcal{X}^\infty$ (does the next symbol match?); the real power of Theorem 3.9.5 is for events that depend on the *infinite future* — essential for far-sighted agents making long-term plans (Chapter 7 and beyond).

Exercises I

1. Prove the elementary relations between a_t, s_t, h_t in Theorem 3.2.3 (third line); show the constants are tight.
2. Prove the ratio convergence $\xi_t/\mu_t \rightarrow 1$ in Corollary 3.3.2; show the *difference* (unlike the ratio) may fail to converge off-sequence.
3. Prove the Hellinger bound, Theorem 3.2.6(iii) (hint: $\sum_i \sqrt{p_i q_i} \leq 1 - \frac{1}{2} \sum_i (\sqrt{p_i} - \sqrt{q_i})^2 \leq \exp[-\frac{1}{2} \sum_i (\sqrt{p_i} - \sqrt{q_i})^2]$).
4. Show that Markov's inequality applied to Corollary 3.3.3 and Theorem 3.3.4 gives the claimed high-probability bound on ε -errors.
5. Prove the high-probability bound on $\sum_t h_t$ in Theorem 3.3.4.
6. Let $\mathcal{M} = \{\mu, \nu\}$ with $\mu = \text{Bern}(\frac{1}{2})$ i.i.d., and $\nu(x_t=1) = \frac{1}{2} - \frac{1}{t+2}$ (not i.i.d.), equal prior $w_\mu = w_\nu = \frac{1}{2}$. Show $w(\mu|x_{<t}) \not\rightarrow 1$ w. μ .p.1; why does this not contradict Corollary 3.3.2?
7. Show all results of Section 3.5 remain valid for $x_{<t}$ -dependent loss; give an example where a non-greedy choice of $y_1 \neq y_1^{\wedge \mu}$ leads to smaller total loss.
8. Write code implementing Λ_ξ for Example 3.5.3 (as we did above); how many iterations, in expectation, until Λ_ξ recommends dropping the jacket?
9. Show $\text{Loss}_t(\Lambda_\rho)$ is continuous in $\rho(x'_t|x_{<t})$ at $\rho = \mu$, but discontinuous at $\rho \neq \mu$.
10. Verify, following Example 3.5.10, that $\lim_{n \rightarrow \infty} \text{Loss}_{1:n} = \infty$.

Exercises II

11. For two mixtures $\xi_w, \xi_{w'}$ with different weight functions, show the μ -expected squared distance between them is upper bounded by $2(\ln w_\mu^{-1} + \ln w_\mu'^{-1})$.
12. Under what conditions on μ can Λ_μ achieve 0 loss?
13. Let $\mathcal{M} = \{\nu_\theta : \theta \in [0, 1]\}$ i.i.d.; show ξ recovers the Laplace rule of Section 2.4.3.
14. Prove that ξ is Pareto optimal w.r.t. some but not all of the distance measures of Definition 3.2.1.
15. Show the mixture ξ is Balanced Pareto optimal.
16. Show Balanced Pareto optimality implies Pareto optimality.
17. Consider the MAP/MDL estimator $\varrho(x) := \max\{w_\nu \nu(x) : \nu \in \mathcal{M}\}$; show it converges in mean² sum but exponentially worse than ξ [Hut05b].
18. Show $\mathbf{P}[\sum_{t=1}^n (\mu_t - \xi_t)^2 \geq \varepsilon^{-1} \ln w_\mu^{-1}] \leq \varepsilon$ and similarly for loss differences; can a similar bound be proved for the ratio $\text{Loss}_t(\Lambda_\xi)/\text{Loss}_t(\Lambda_\mu)$?
19. Show/refute that if s_t were monotone decreasing, then $\mathbf{E}_\mu[s_t] = o(1/t)$; give (necessary/sufficient) conditions on $\mathcal{M}$ under which this monotonicity generally fails.
20. List reasons why we might (not) want to restrict $\mathcal{M}$ to (semi)computable (semi)measures; derive non-computable (semi)measures of interest.

Exercises III

21. Prove $\mathcal{M}_{comp}$ is not computably enumerable and its mixture is not even semicomputable, but is approximable; prove $\mathcal{M}_{sol}$ is computably enumerable, and ξ_U is lower semicomputable but not computable.
22. Prove that for any choice of weights w_ν on $\mathcal{M}_{sol}$, $\xi(x) := \sum_{\nu \in \mathcal{M}_{sol}} w_\nu \nu(x) \stackrel{x}{\geq} w_\nu^U \nu(x)$; how large is the hidden multiplicative constant?

History and References I

Origins. Solomonoff induction combines algorithmic information theory with Bayesian reasoning, and represents a platonic ideal for inductive reasoning. Introductory surveys: [HLV07, Hut17] (short, encyclopedic); [Hut09b, Sec.2] (further insights); [RH11] (long, gentle, philosophical); [Hut06c, Hut07e] (mathematical, in-depth). Optimality of Solomonoff induction: [Hut03e]. General loss bounds: [Hut03a, Hut01c]. Research on Solomonoff induction was revived by [Hut01d, Hut01a] with extension to non-binary alphabet and 0-1 loss bounds.

Sharper future bounds. [CH05, CHS07] generalize Solomonoff's bound to a *future* bound in terms of $K(\mu|x_{<t})$ (which can be significantly smaller than $K(\mu)$, since it already “knows” the history). [Leg06] shows any powerful predictor must itself be complex. In the context of the black raven paradox, [LH15e] observes Solomonoff induction violates Nicod's criterion, but blames the criterion, not induction. Solomonoff induction is unified with decision theory to give the **AIXI** agent [Hut00, Hut07f, Hut05b] (Chapter 7).

History and References II

Computability nuances. [LH15d] categorizes the computability of various versions of Solomonoff's prior via the arithmetic hierarchy. [LHS13a] derives tight probability bounds on the cumulative error. [WSH11] compares and contrasts different choices for M and w_ν . [Hut03b, Hut06b] explore alternative *computable* priors. [LHG11] shows Solomonoff induction may fail on incomputable but “partially computable” sequences (e.g. halting-problem indicator sequences interleaved with zeros); a *normalized* version [HM04, HM07, LH13, LH15a] fixes this, at the cost of failing on Martin-Löf random sequences (resolving open problem [Hut03c]). [Hut09g] lists many further open problems.

K -complexity as difficulty measure. As Theorem 3.7.3 among others show, K -complexity is a good measure of the intrinsic difficulty of a sequence prediction problem. [AEH14] measures the difficulty of (open-box) optimization problems similarly.

History and References III

Alternatives to Bayesian mixtures. The **Minimum Description Length (MDL)** principle — optimize for a model that compresses data plus model as much as possible (Section 2.7.4) — has comparable performance to Bayes for continuously parameterized i.i.d. classes [Gru07], and is asymptotically consistent for general non-i.i.d. sequence prediction [Hut09b], though its convergence *rate* can be exponentially worse [PH04b, PH06a, Hut03g, Hut06d]. **Prediction with expert advice** [HP04, HP05] is another alternative, tuned to a particular loss function [Hut04]; [RH07, RH08b, Rya20] studies when mixture predictors exist that predict *all* other measures for ultra-general classes.

Smaller model classes and applications. Chapters 4 and 5 introduce model classes smaller than $\mathcal{M}_{comp}$ for which ξ is efficiently computable. Chapter 7 covers extensions to sequential decision making and reinforcement learning. Recent work [GMGH+24] trains neural networks to mimic Solomonoff induction on approximate-Solomonoff-sampled training data; [DRD+24] shows large language models are themselves strong (lossless) compressors, even for images, despite being trained primarily on text.

Broader Bayesian statistics. For general and non-parametric Bayesian inference, see [BC16, Pre09, Lee12, Jay03, GCSR95, GvdV17]. Bayesian credible sets [Hut08a], the analytic Bayesian posterior of

History and References IV

mutual information [Hut02a, ZH02, HZ05], and Bayesian handling of missing data [HZ03] round out the chapter's broader statistical context.

Summary: Chapter 3

- The **Bayes mixture** $\xi = \sum_{\nu} w_{\nu} \nu$ (Def. 3.1.2) **dominates** every model in its class ($\xi \geq w_{\nu} \nu$, Prop. 3.1.4), with no i.i.d. or stationarity assumptions needed.
- The **Generalized Solomonoff bound** $D_n \leq \ln w_{\mu}^{-1}$ (Thm. 3.2.5) is the central inequality: total KL error is bounded by a constant that never grows with n , giving **predictive convergence** $\xi \rightarrow \mu$ (Sec. 3.3) even under **model misspecification** (Sec. 3.4).
- The Bayes-optimal predictor Λ_{ξ} suffers only $O(\sqrt{\text{Loss}_{1:n}(\Lambda_{\mu})})$ **excess loss** relative to the ideal Λ_{μ} (Sec. 3.5), and ξ is provably **Pareto optimal** — no predictor can beat it on one environment without doing worse on another (Sec. 3.6).
- Choosing $\mathcal{M} = \mathcal{M}_{sol}$ and the **Solomonoff prior** $w_{\nu}^U = 2^{-K(\nu)}$ (Sec. 3.7) gives the tightest possible bounds, grounded in Kolmogorov complexity and a rigorous “proof” of Occam’s razor.
- The **Solomonoff distribution** M (Sec. 3.8), built from a universal Turing machine fed random noise, is remarkably equivalent to ξ_U — and both come with strong predictive guarantees.
- **Martingale theory** (Sec. 3.9) gives an alternative, assumption-light route to convergence, culminating in the powerful *merging of opinions* theorem for arbitrary future events.
- This machinery underlies the **AIXI** agent (Chapter 7) and recurs throughout the rest of the book.